Mid-Semester Exam

DDD 2: Introduction to Programming and Basic Data Analysis

Ramesh Beniwal | Admission ID: BAEC2240006

2026-11-03

Table of contents

Setup 1
Load Required Libraries 1
Exam Environment 2
Question 1 2
Question 2 3
Question 3 5
Question 4 7
Question 5 8
Question 6 8
Question 7 10
Final Check 11

Setup

```
rm(list = ls())
```

Load Required Libraries

```
#Library loading
library(tibble)
library(dplyr)
library(rlang)
library(lobstr)
library(rio)
```

Exam Environment

```
# All exam answers and variables
MIDSEM_DDD <- new.env()
```

Question 1

```
# Step 1: Create a character
# c() creating  multiple vectors
MIDSEM_DDD$performance <- c("Poor", "Average", "Good", "Excellent",
                            "Good", "Poor", "Excellent", "Average")
# Step 2: Convert to an ordered factor
MIDSEM_DDD$perf_factor <- factor(MIDSEM_DDD$performance,
                           levels = c("Poor", "Average", "Good", "Excellent"),
                                ordered = TRUE)
# Step 3: Print the factor to see categories and their order
MIDSEM_DDD$perf_factor
```

```
[1] Poor      Average   Good      Excellent Good      Poor      Excellent
[8] Average
Levels: Poor < Average < Good < Excellent
```

```
# Step 4: Verify the internal representation
as.numeric(MIDSEM_DDD$perf_factor)
```

```
[1] 1 2 3 4 3 1 4 2
```

```
typeof(MIDSEM_DDD$perf_factor)
```

```
[1] "integer"
```

```
levels(MIDSEM_DDD$perf_factor) #shows the order of categories
```

```
[1] "Poor"      "Average"   "Good"      "Excellent"
```

```
str(MIDSEM_DDD$perf_factor) # shows the full structure of the factor
```

```
 Ord.factor w/ 4 levels "Poor"<"Average"<..: 1 2 3 4 3 1 4 2
```

Question 2

```
# Step 1 Creatig a data frame as per the question
MIDSEM_DDD$df <- tibble(  # using tibble to create a data frame
  individual_id      = c(1, 2, 3, 4, 5),
  years_of_education = c(10, 14, 12, 16, 8),
  employment_status  = c("Employed", "Unemployed",
                         "Employed", "Employed",
                         "Unemployed"),
  monthly_income     = c(25000, 0, 18000, 45000, 0)
)
print(MIDSEM_DDD$df)
```

```
# A tibble: 5 × 4
  individual_id years_of_education employment_status monthly_income
          <dbl>              <dbl> <chr>                       <dbl>
1             1                 10 Employed                    25000
2             2                 14 Unemployed                      0
3             3                 12 Employed                    18000
4             4                 16 Employed                    45000
5             5                  8 Unemployed                      0
```

```
# ------------------------------------------------
# Part (a): Rename variables to better names
# rename() changes column names
# new_name = old_name is the syntax
# ------------------------------------------------

MIDSEM_DDD$df <- MIDSEM_DDD$df %>%
  rename(
    edu_years      = years_of_education,   # shorter and cleaner name
    income_monthly = monthly_income        # clearer name
  )

print(MIDSEM_DDD$df)
```

```
# A tibble: 5 × 4
  individual_id edu_years employment_status income_monthly
```

```
          <dbl>     <dbl> <chr>                       <dbl>
1             1        10 Employed                    25000
2             2        14 Unemployed                      0
3             3        12 Employed                    18000
4             4        16 Employed                    45000
5             5         8 Unemployed                      0
```

```
# ------------------------------------------------
# Part (b): Create new variable - more than 12 years of education?
# mutate() adds a new column to the data frame
# ifelse(condition, value_if_true, value_if_false)
# ------------------------------------------------

MIDSEM_DDD$df <- MIDSEM_DDD$df %>%
  mutate(
    high_edu = ifelse(edu_years > 12, TRUE, FALSE)
  )
print(MIDSEM_DDD$df)
```

```
# A tibble: 5 × 5
  individual_id edu_years employment_status income_monthly high_edu
          <dbl>     <dbl> <chr>                       <dbl> <lgl>
1             1        10 Employed                    25000 FALSE
2             2        14 Unemployed                      0 TRUE
3             3        12 Employed                    18000 FALSE
4             4        16 Employed                    45000 TRUE
5             5         8 Unemployed                      0 FALSE
```

```
# ------------------------------------------------
# Part (c): Subset - keep only individuals with high_edu == TRUE
# filter() keeps only rows that match the condition
# ------------------------------------------------

MIDSEM_DDD$df_subset <- MIDSEM_DDD$df %>%
  filter(high_edu == TRUE)
print(MIDSEM_DDD$df_subset)
```

```
# A tibble: 2 × 5
  individual_id edu_years employment_status income_monthly high_edu
          <dbl>     <dbl> <chr>                       <dbl> <lgl>
```

```
1             2        14 Unemployed                    0 TRUE
2             4        16 Employed                  45000 TRUE
```

```
# ------------------------------------------------
# Part (d): Keep only individual_id and high_edu columns
# select() chooses which columns to keep
# ------------------------------------------------

MIDSEM_DDD$df_final <- MIDSEM_DDD$df_subset %>%
  select(individual_id, high_edu)
print(MIDSEM_DDD$df_final)
```

```
# A tibble: 2 × 2
  individual_id high_edu
          <dbl> <lgl>
1             2 TRUE
2             4 TRUE
```

Question 3

```
# Step 1: Create data frame with 4 numeric variables
MIDSEM_DDD$df3 <- data.frame(
  a = c(10, 20, 30, 40, 50, 60),
  b = c(5, 15, 25, 35, 45, 55),
  c = c(2, 4, 6, 8, 10, 12),
  d = c(1, 3, 5, 7, 9, 11)
)
print(MIDSEM_DDD$df3)
```

```
   a  b  c  d
1 10  5  2  1
2 20 15  4  3
3 30 25  6  5
4 40 35  8  7
5 50 45 10  9
6 60 55 12 11
```

```
# ------------------------------------------------
# Part (a): Replace some elements with NA (missing values)
```

```r
# [row, column] targets a specific cell in the data frame
# ------------------------------------------------

MIDSEM_DDD$df3[2, "a"] <- NA
MIDSEM_DDD$df3[4, "b"] <- NA
MIDSEM_DDD$df3[1, "c"] <- NA    # row 1 of column c becomes missing
MIDSEM_DDD$df3[3, "d"] <- NA

print(MIDSEM_DDD$df3)
```

```
   a  b  c  d
1 10  5 NA  1
2 NA 15  4  3
3 30 25  6 NA
4 40 NA  8  7
5 50 45 10  9
6 60 55 12 11
```

```r
# ------------------------------------------------
# Part (b): Write Z-score function
# Z-score formula: (value - mean) / standard deviation
# na.rm = TRUE means ignore NA values during calculation
# ------------------------------------------------

MIDSEM_DDD$z_score <- function(x) {
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
} # x is the input numeric vector
  # mean(x, na.rm=TRUE) calculates average, skipping NA values (same for sd also)


# ------------------------------------------------
# Part (c): Apply function to transform the data frame
# mutate() creates or changes columns
# ------------------------------------------------

MIDSEM_DDD$df3_change <- MIDSEM_DDD$df3 %>%
  mutate(
    # Replace columns a, b, c with their z-scores
    across(c(a, b, c), MIDSEM_DDD$z_score),

    # New variable mean_deviation = each value of d minus the mean of d
    mean_deviation = d - mean(d, na.rm = TRUE),
```

```
    # New variable sum_z = sum of z-scores of a, b, c for each row
    # rowSums() adds values across columns row by row
    sum_z = rowSums(across(c(a, b, c)), na.rm = TRUE)
  )

print(MIDSEM_DDD$df3_change)
```

```
           a          b          c  d mean_deviation     sum_z
1 -1.4556507 -1.1573828         NA  1           -5.2 -2.613033
2         NA -0.6751400 -1.2649111  3           -3.2 -1.940051
3 -0.4159002 -0.1928971 -0.6324555 NA             NA -1.241253
4  0.1039750         NA  0.0000000  7            0.8  0.103975
5  0.6238503  0.7715885  0.6324555  9            2.8  2.027894
6  1.1437255  1.2538313  1.2649111 11            4.8  3.662468
```

Question 4

```
# Step 1: Create a numeric vector
MIDSEM_DDD$vec <- c(4, 8, 15, 16, 23, 42)

# Step 2: Calculate the mean of the full vector ONCE before the loop
MIDSEM_DDD$vec_mean <- mean(MIDSEM_DDD$vec)   # calculates the average

# Step 3: Write the absolute deviation function
MIDSEM_DDD$abs_dev <- function(x, mean_val) {
  sqrt((x - mean_val)^2)
}

# Step 4: Preallocate empty output vector
# numeric(length(vec)) creates a vector of zeros, same size as vec
MIDSEM_DDD$result4 <- numeric(length(MIDSEM_DDD$vec))

# Step 5: Apply function element by element using for loop
# i goes from 1 to length of vec
for (i in 1:length(MIDSEM_DDD$vec)) {
         MIDSEM_DDD$result4[i]     <-     MIDSEM_DDD$abs_dev(MIDSEM_DDD$vec[i],
MIDSEM_DDD$vec_mean)
}
print(MIDSEM_DDD$result4)
```

```
[1] 14 10  3  2  5 24
```

```r
# ------------------------------------------------
# WITHOUT for loop - same result in one line
# ------------------------------------------------
# sqrt() of squared difference gives absolute deviation
MIDSEM_DDD$result4_no_loop <- sqrt((MIDSEM_DDD$vec - mean(MIDSEM_DDD$vec))^2)
print(MIDSEM_DDD$result4_no_loop)
```

```
[1] 14 10  3  2  5 24
```

Question 5

```r
# We need two vectors for two loops
MIDSEM_DDD$outer_vec <- c(2, 4, 6)    # outer values
MIDSEM_DDD$inner_vec <- c(10, 20)     # inner values

# Nested for loop:
# For each i in (2, 4, 6), loop through all j in (10, 20)
for (i in MIDSEM_DDD$outer_vec) {
  for (j in MIDSEM_DDD$inner_vec) {
    print(c(i, j))
  }
}
```

```
[1]  2 10
[1]  2 20
[1]  4 10
[1]  4 20
[1]  6 10
[1]  6 20
```

Question 6

```r
# Create data frame with 5 employees
MIDSEM_DDD$df6 <- tibble(
  id     = c(1, 2, 3, 4, 5),
  salary = c(20000, 35000, 28000, 42000, 31000)
)
```

```r
# hike = fixed amount added to every employee salary
MIDSEM_DDD$hike <- 5000

# ------------------------------------------------
# PART (a) - Create empty vector FIRST before loop
# storing length in a simple variable n
# ------------------------------------------------
n <- length(MIDSEM_DDD$df6$salary)   # n = 5
new_salary <- numeric(n)              # creates --> 0 0 0 0 0

# ------------------------------------------------
# PART (b) - For loop to calculate new salary
# ------------------------------------------------
for (i in 1:n) {
  new_salary[i] <- MIDSEM_DDD$df6$salary[i] + MIDSEM_DDD$hike
}
new_salary
```

```
[1] 25000 40000 33000 47000 36000
```

```r
# ------------------------------------------------
# PART (c) - Add new_salary as a column in df6
# ------------------------------------------------
MIDSEM_DDD$df6$new_salary <- new_salary
MIDSEM_DDD$df6
```

```
# A tibble: 5 × 3
     id salary new_salary
  <dbl>  <dbl>      <dbl>
1     1  20000      25000
2     2  35000      40000
3     3  28000      33000
4     4  42000      47000
5     5  31000      36000
```

```r
# ------------------------------------------------
# PART (d) - WITHOUT for loop
# ------------------------------------------------
MIDSEM_DDD$df6$new_salary_v2 <- MIDSEM_DDD$df6$salary + MIDSEM_DDD$hike
MIDSEM_DDD$df6
```

```
# A tibble: 5 × 4
     id salary new_salary new_salary_v2
  <dbl>  <dbl>      <dbl>         <dbl>
1     1  20000      25000         25000
2     2  35000      40000         40000
3     3  28000      33000         33000
4     4  42000      47000         47000
5     5  31000      36000         36000
```

Question 7

```
# Step 1: Create a vector of marks for 10 students
MIDSEM_DDD$marks <- c(95, 82, 67, 55, 78, 91, 60, 45, 73, 88)

# Step 2: Preallocate empty character vector to store grades
MIDSEM_DDD$grades <- character(length(MIDSEM_DDD$marks))

# Step 3: For loop with if-else to check each student's marks
for (i in 1:length(MIDSEM_DDD$marks)) {

  if (MIDSEM_DDD$marks[i] >= 90) {   # marks >= 90 gets Grade A
    MIDSEM_DDD$grades[i] <- "Grade A"

  } else if (MIDSEM_DDD$marks[i] >= 75) { # marks >= 75 but less than 90 gets
Grade B
    MIDSEM_DDD$grades[i] <- "Grade B"

  } else if (MIDSEM_DDD$marks[i] >= 60) {    # marks >= 60 but less than 75 gets
Grade C
    MIDSEM_DDD$grades[i] <- "Grade C"

  } else {      # marks below 60 gets Fail
    MIDSEM_DDD$grades[i] <- "Fail"
  }
}


print(MIDSEM_DDD$grades)
```

```
 [1] "Grade A" "Grade B" "Grade C" "Fail"    "Grade B" "Grade A" "Grade C"
 [8] "Fail"    "Grade C" "Grade B"
```

```r
# Step 5: Show marks and grades together
MIDSEM_DDD$result_q7 <- tibble(
  marks  = MIDSEM_DDD$marks,
  grades = MIDSEM_DDD$grades
)

print(MIDSEM_DDD$result_q7)
```

```
# A tibble: 10 × 2
   marks grades
   <dbl> <chr>
 1    95 Grade A
 2    82 Grade B
 3    67 Grade C
 4    55 Fail
 5    78 Grade B
 6    91 Grade A
 7    60 Grade C
 8    45 Fail
 9    73 Grade C
10    88 Grade B
```

Final Check

```r
ls(MIDSEM_DDD)
```

```
 [1] "abs_dev"         "df"              "df_final"        "df_subset"
 [5] "df3"             "df3_change"      "df6"             "grades"
 [9] "hike"            "inner_vec"       "marks"           "outer_vec"
[13] "perf_factor"     "performance"     "result_q7"       "result4"
[17] "result4_no_loop" "vec"             "vec_mean"        "z_score"
```